o2h Technology Interview Prep: 50 DSA + 100 Full Stack Questions

Prepared for an interview at o2h Technology (https://o2htechnology.com/), a software and AI services company based in Ahmedabad (India) and Cambridge (UK). They build products for startups, SMEs and enterprises, with a focus on life sciences. Their stack includes React, Angular, Vue, Node.js, Python, Java, PHP, PostgreSQL, MySQL, MongoDB, AWS/Azure/GCP, Docker/Kubernetes, and LLM tooling (OpenAI, Anthropic, RAG, vector databases such as Pinecone and FAISS). The questions below are weighted toward that stack. This is preparation material, not their actual question bank.

Part 1: Top 50 DSA Questions

Arrays and Strings

1. Two Sum
2. Best Time to Buy and Sell Stock
3. Contains Duplicate
4. Product of Array Except Self
5. Maximum Subarray (Kadane's)
6. Merge Intervals
7. Rotate Array
8. Move Zeroes
9. Longest Substring Without Repeating Characters
10. Valid Anagram
11. Group Anagrams
12. Longest Palindromic Substring
13. Valid Parentheses
14. Trapping Rain Water
15. 3Sum

Searching, Sorting, Two Pointers

16. Binary Search and its variants

17. Search in Rotated Sorted Array
18. Find First and Last Position of Element
19. Merge Two Sorted Arrays
20. Kth Largest Element in an Array

Linked Lists

21. Reverse a Linked List
22. Detect a Cycle (Floyd's)
23. Merge Two Sorted Lists
24. Remove Nth Node From End
25. Find the Middle of a Linked List

Stacks and Queues

26. Min Stack
27. Implement Queue Using Stacks
28. Next Greater Element
29. Sliding Window Maximum
30. LRU Cache

Trees and Graphs

31. Maximum Depth of Binary Tree
32. Level Order Traversal
33. Validate Binary Search Tree
34. Lowest Common Ancestor
35. Invert Binary Tree
36. Serialize and Deserialize Binary Tree
37. Number of Islands
38. Clone Graph
39. Course Schedule (cycle detection, topological sort)
40. Shortest Path in a Graph (BFS/Dijkstra)

Dynamic Programming

41. Climbing Stairs
42. House Robber
43. Coin Change
44. Longest Increasing Subsequence
45. 0/1 Knapsack
46. Longest Common Subsequence

Recursion, Backtracking, Heaps, Hashing

47. Subsets and Permutations
48. Top K Frequent Elements
49. Merge K Sorted Lists
50. Word Search / N-Queens

Part 2: Top 100 Full Stack Questions

JavaScript and TypeScript (1-15)

1. Explain closures with a practical example.
2. What is the difference between `var`, `let`, and `const`?
3. How does the event loop work? Explain the microtask and macrotask queues.
4. What is hoisting?
5. Explain `this` and how `call`, `apply`, and `bind` change it.
6. Promises vs async/await vs callbacks.
7. What is prototypal inheritance?
8. Debouncing vs throttling. Implement each.
9. Shallow copy vs deep copy.
10. What are `map`, `filter`, and `reduce`? When would you use each?
11. `==` vs `===`, and type coercion pitfalls.
12. What are ES6+ features you use daily (destructuring, spread, modules)?
13. What is the difference between `null`, `undefined`, and `NaN`?
14. TypeScript: interface vs type, generics, and `unknown` vs `any`.
15. How do you handle errors in async code?

React (16-30)

16. Virtual DOM and reconciliation.
17. Props vs state.
18. Explain `useEffect` and its dependency array.
19. `useMemo` vs `useCallback` vs `React.memo`.
20. When would you use `useReducer` over `useState`?
21. Controlled vs uncontrolled components.
22. Why do lists need keys?
23. Context API vs Redux. When is each appropriate?
24. How do you optimize a slow React app?
25. Code splitting and lazy loading.
26. Custom hooks: write one (e.g., `useFetch`).
27. What causes unnecessary re-renders and how do you prevent them?
28. Server-side rendering vs client-side rendering vs static generation.
29. How do you handle forms and validation?
30. How do you test React components (Jest, React Testing Library)?

HTML, CSS, Browser (31-38)

31. Semantic HTML and why it matters.
32. The CSS box model.
33. Flexbox vs Grid.
34. How do you build a responsive layout?
35. CSS specificity rules.
36. `position`: static, relative, absolute, fixed, sticky.
37. What happens when you type a URL and press Enter?
38. localStorage vs sessionStorage vs cookies.

Backend: Node.js, Python, Java, PHP (39-53)

39. How does Node.js handle concurrency if it is single-threaded?
40. What is middleware in Express?
41. Explain streams and buffers in Node.js.
42. `process.nextTick` vs `setImmediate`.

43. How do you structure a production Express or Node project?
44. How do you handle file uploads securely?
45. Flask vs Django vs FastAPI. When would you choose each?
46. Python: GIL, decorators, generators, list comprehensions.
47. Java: OOP pillars, `HashMap` internals, and `equals` / `hashCode`.
48. Java: checked vs unchecked exceptions, and the Spring Boot basics.
49. PHP: how does Laravel handle routing, middleware, and Eloquent?
50. Synchronous vs asynchronous processing. When would you use a queue?
51. How do you implement background jobs and cron tasks?
52. How do you implement pagination, filtering, and sorting in an API?
53. How do you handle logging and monitoring in production?

APIs and Architecture (54-65)

54. REST principles and HTTP methods. What does idempotent mean?
55. Common HTTP status codes (200, 201, 204, 400, 401, 403, 404, 409, 429, 500).
56. REST vs GraphQL vs gRPC.
57. How do you version an API?
58. PUT vs PATCH.
59. How do you implement rate limiting?
60. Monolith vs microservices.
61. What is CORS and how do you fix a CORS error?
62. Webhooks vs polling vs WebSockets vs Server-Sent Events.
63. How do you design a URL shortener?
64. How do you design a notification system?
65. How do you design a file upload and storage service?

Authentication and Security (66-75)

66. Session-based vs token-based (JWT) authentication.
67. How do access and refresh tokens work?
68. OAuth 2.0 and OpenID Connect basics.
69. How do you hash passwords (bcrypt, argon2)? Why not plain SHA?
70. XSS: what is it and how do you prevent it?

71. CSRF: what is it and how do you prevent it?
72. SQL injection and prevention.
73. What is HTTPS/TLS and why does it matter?
74. Role-based access control: how do you implement it?
75. How do you store secrets and API keys safely?

Databases (76-90)

76. SQL vs NoSQL. When would you choose MongoDB over PostgreSQL?
77. ACID properties.
78. Types of joins with examples.
79. What are indexes, and when can they hurt performance?
80. Normalization vs denormalization.
81. Write a query for the second-highest salary.
82. `GROUP BY` vs `HAVING`, and `WHERE` vs `HAVING`.
83. What are transactions and isolation levels?
84. What is the N+1 query problem and how do you fix it?
85. How do you analyze a slow query (`EXPLAIN`)?
86. MongoDB: aggregation pipeline, embedding vs referencing.
87. Replication vs sharding.
88. What is database migration and how do you manage it?
89. CAP theorem.
90. When and how would you use Redis for caching?

Cloud, DevOps, Testing (91-97)

91. Explain Docker images vs containers, and write a basic Dockerfile.
92. What does Kubernetes do (pods, deployments, services)?
93. What is a CI/CD pipeline? Describe one you have built (Jenkins, GitHub Actions).
94. Core AWS services you have used (EC2, S3, RDS, Lambda, IAM).
95. How do you deploy with zero downtime?
96. Unit vs integration vs end-to-end testing.
97. Git: merge vs rebase, resolving conflicts, and branching strategy.

Gen AI and LLM Integration (98-100)

Relevant because o2h offers Gen AI and AI agent development services.

98. What is RAG, and how would you build a document Q&A app?
99. What are embeddings and vector databases (Pinecone, FAISS, Chroma)? How do you choose between them?
100. How do you reduce hallucinations, control cost and latency, and handle prompt injection when calling LLM APIs?

Preparation Tips

- **Be ready to discuss your own projects in depth.** At service companies like this, interviewers often spend more time on "what did you build and what were the trade-offs?" than on trivia.
- **For DSA, practice explaining your approach aloud** before coding, and state time and space complexity.
- **Look at their case studies** (CITC, Stamp Free, miPatient, CellRepo) so you can ask informed questions about their products.
- **Expect agile and client-communication questions**, since they work in sprints with startup clients: how you handle changing requirements, estimate tasks, and report blockers.